

# Kubernetes Beginner Guide

## Chapter 2: Kubernetes Core Objects

Explained in simple English, figure by figure. Theory only, no code.

### What you will learn

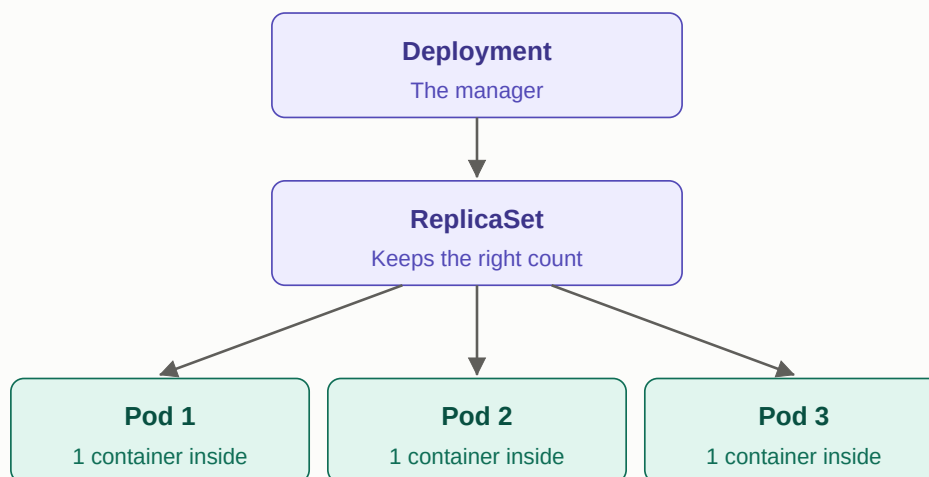
- What a **Pod**, **ReplicaSet** and **Deployment** are, and how they work together
- How Kubernetes heals itself and updates apps with no downtime
- What **Namespaces** are and why teams use them
- How **labels and selectors** connect objects
- What **YAML manifests** and **declarative configuration** mean

### How to read this chapter

Each topic has one figure. First look at the figure, then read the explanation under it. The explanation walks through every part of the picture, gives the reason why it exists, and ends with a real-life example. There are six figures in total.

## 1. The big picture: how the objects are connected

You will mostly work with four things: **Deployment**, **ReplicaSet**, **Pod** and **container**. They sit inside each other like layers. You create the top layer yourself, and it creates the layers below it.



*Figure 1. A Deployment creates a ReplicaSet, and the ReplicaSet creates the Pods.*

### Explaining Figure 1

The arrows mean *creates and manages*. The top layer is in charge, and each layer below does one smaller job.

**Deployment (top, purple).** This is the object you create yourself. You tell it: "I want 3 copies of my app, using this image." It does not run the app. It is a manager that creates a ReplicaSet and watches over it. It also handles updates, scaling and rollback (see Figure 3).

**ReplicaSet (middle, purple).** The Deployment creates it automatically, so you almost never create one yourself. Its only job is to keep the number of Pods correct. If you want 3 Pods, it makes sure 3 are always alive. The three arrows going down mean "I created these Pods and I watch them".

**Pods (bottom, teal).** Pods are where your app really runs. Each Pod is a box that holds one container, which is your packaged app. The three Pods are identical copies, so if one is busy, the others can serve users. Pods are temporary and can be replaced at any time (see Figure 2).

### Why so many layers?

Each layer solves one problem: Pods run the app, ReplicaSets keep the count right, and Deployments make updates safe. If everything was in one object, updating an app without stopping it would be very hard.

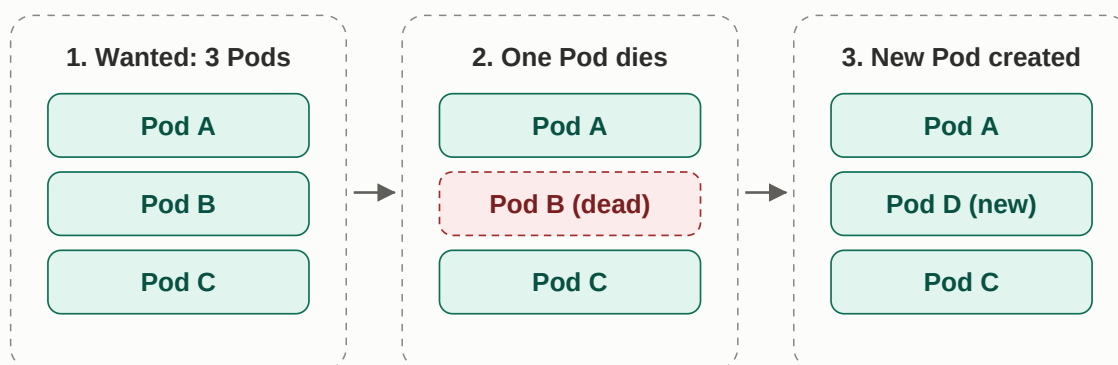
### Why Pods and not containers directly?

Kubernetes needs one standard unit to manage. A Pod gives its containers one IP address, shared storage, and a shared start and stop. In real work, most Pods hold only one container. Sometimes a Pod holds a second helper container, for example one that collects logs.

## 2. Pod and ReplicaSet: self-healing

**Pod** is the smallest thing Kubernetes can run. The most important rule: Pods are temporary, like paper cups. When a Pod dies it is not repaired. A new Pod is created, with a new name and a new IP address.

**ReplicaSet** is a watcher that keeps a fixed number of identical Pods running. A single Pod that dies stays dead, so the ReplicaSet fixes this by always comparing "how many do I want?" with "how many do I have?".



*Figure 2. A ReplicaSet notices a missing Pod and creates a new one.*

### Explaining Figure 2

Read the figure from left to right. It is a story in three steps.

**Panel 1: Wanted 3 Pods.** You asked for 3 Pods, and 3 are running: A, B and C. Everything is healthy. The ReplicaSet watches all the time, like a security guard counting people at a door.

**Panel 2: One Pod dies.** Pod B crashes. The reasons can be a bug, running out of memory, or the whole machine (node) failing. Only 2 Pods are alive now, but users are still served by A and C, so the app does not fully go down. The ReplicaSet notices: "I want 3, but I have 2."

**Panel 3: A new Pod is created.** The ReplicaSet creates a new Pod right away. The Scheduler chooses a node for it, and it starts running. We are back to 3 Pods, and you did nothing by hand.

### Important details

- The new Pod is called **D**, not B. Kubernetes does not repair the dead Pod. It replaces it with a brand new one, with a new name and a new IP address.
- So never depend on one specific Pod. Later, a Service (Chapter 3) will give you one stable address in front of all Pods.
- Never keep important data inside a Pod. When the Pod is replaced, its data is gone. Chapter 4 (Storage) solves this.
- The idea behind this is **desired state versus actual state**. Desired is 3, actual is 2, and Kubernetes works until they match.

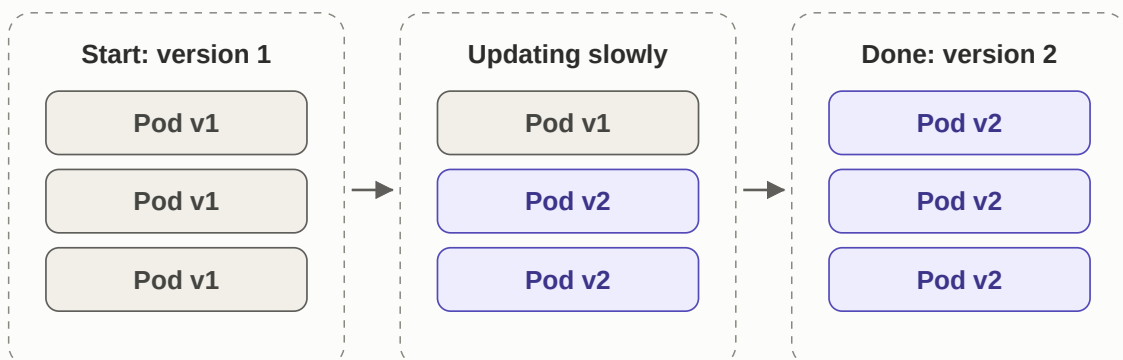
#### Real-life example

A shop needs 3 cashiers all day. If one cashier goes home sick, the manager calls a replacement so the shop always has 3. The new person is a different person, but the job is the same.

## 3. Deployment: rolling updates and rollback

A ReplicaSet only keeps the count right. A **Deployment** is the object you use most in real work, because it manages ReplicaSets and adds four extra powers:

| Power          | Meaning                                            |
|----------------|----------------------------------------------------|
| Scaling        | Change from 3 Pods to 10 Pods, or back to 2        |
| Rolling update | Move to a new app version slowly, with no downtime |
| Rollback       | If the new version is bad, go back to the old one  |
| Self-healing   | Replace dead Pods (through the ReplicaSet)         |



**Figure 3.** A rolling update replaces old Pods (gray) with new Pods (purple) one by one.

## Explaining Figure 3

**Panel 1: Start (version 1).** All 3 Pods run the old app version and users are happy. Now your team releases version 2 and you want to update.

**Panel 2: Updating slowly.** The Deployment creates a new ReplicaSet for version 2. It starts version 2 Pods one at a time and stops old Pods one at a time. Here, one old Pod remains and two new Pods are running. At every moment some Pods are alive and serving users, so users see no downtime. The Deployment also waits until each new Pod is healthy before removing the next old one. If a new Pod fails to start, the update pauses, so a bad release does not destroy your working app.

**Panel 3: Done (version 2).** All Pods run the new version. The old ReplicaSet is not deleted. It is kept with 0 Pods.

### Why keep the old ReplicaSet? Rollback.

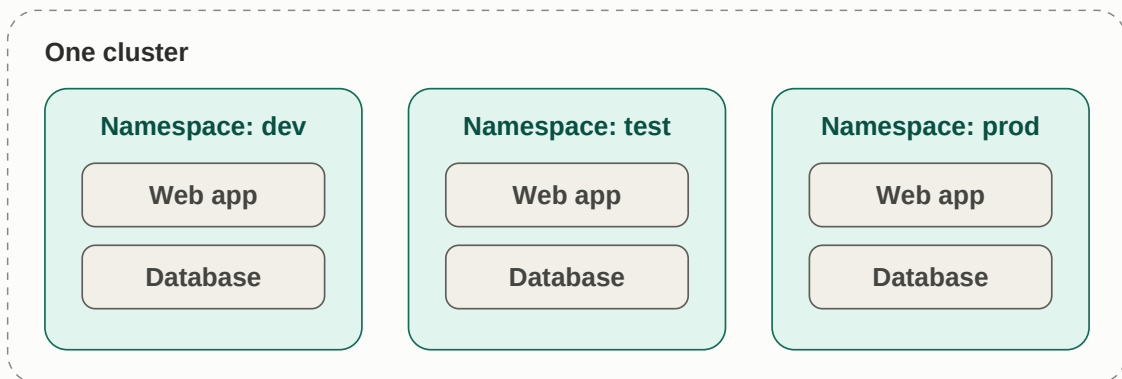
If version 2 has a bug, the Deployment can move Pods back to the old ReplicaSet, and you are on version 1 again quickly. This is called a **rollback**, and it is one of the biggest reasons companies use Deployments.

#### Real-life example

A bus company replaces old buses with new ones. They do not stop all buses on the same day. They replace one bus at a time, so passengers can always travel. If the new bus model has a problem, they bring the old model back.

## 4. Namespace: virtual folders in a cluster

A **Namespace** is a virtual folder inside one cluster. It separates resources into groups. One cluster is often shared by many teams or environments, and Namespaces keep them apart.



*Figure 4. One cluster split into three Namespaces: dev, test and prod.*

## Explaining Figure 4

**The outer dashed box is the cluster.** It is your whole Kubernetes cluster with all its worker nodes. All three Namespaces share the same physical machines, so the separation is logical, not physical.

**The three teal boxes are Namespaces.** *dev* is for developers to try new things, and mistakes there are safe. *test* is for testers to check the app before release. *prod* (production) is the live app that real users use, so it must stay stable.

The gray boxes are objects. Each Namespace has its own web app and its own database. The three web apps have the same name but do not conflict, because names only need to be unique inside one Namespace.

Why we need Namespaces

- 1 **Organization:** neat groups instead of one giant messy list.
- 2 **Safety:** a developer working in dev cannot accidentally delete something in prod.
- 3 **Access control:** you can say "the dev team can only touch dev" (RBAC, Chapter 5).
- 4 **Resource limits:** you can say "dev can use at most 10 CPUs" (Resource Quotas, Chapter 6).

Namespaces that Kubernetes creates for you

| Namespace       | What it holds                                         |
|-----------------|-------------------------------------------------------|
| default         | Your objects, if you do not choose a Namespace        |
| kube-system     | Kubernetes' own parts, such as the API server and DNS |
| kube-public     | Data that everyone can read                           |
| kube-node-lease | Small signals that show each node is alive            |

Real-life example

A big office building has many companies. All share the same building (the cluster), but each company has its own floor (Namespace) and can use the same room names, such as "Meeting Room 1", without confusion. Note: Namespaces separate names and permissions. They do not automatically block network traffic between them. That needs Network Policies (Chapter 5).

5. Labels and selectors: how objects find each other

A **label** is a small tag on an object, written as a key and a value, such as `app = web`. A **selector** is a search rule: "find every object with this label." Kubernetes connects objects with labels, not with names.

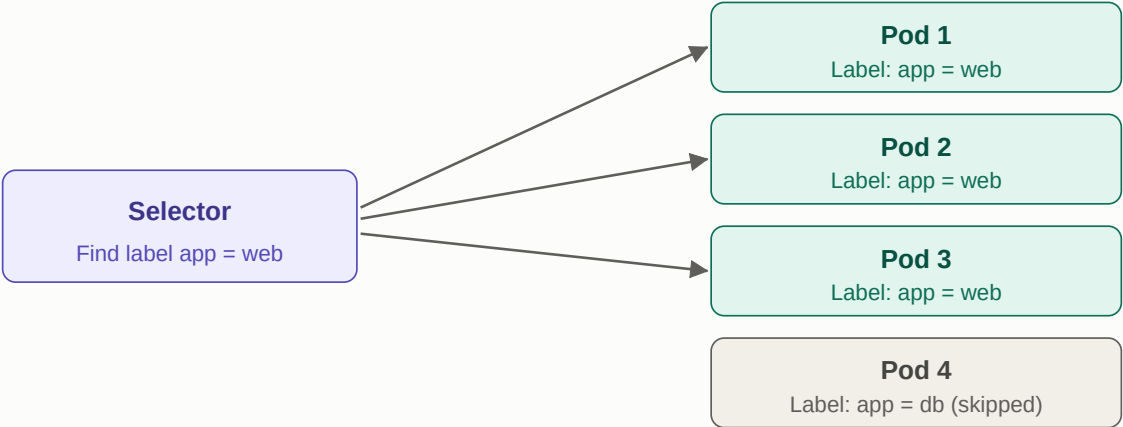


Figure 5. A selector finds all Pods that carry the label `app = web`.

## Explaining Figure 5

**The right side: four Pods with labels.** Pods 1, 2 and 3 carry the label `app = web`, so they are web servers. Pod 4 carries the label `app = db`, so it is a database. You choose the labels yourself. Common ones are `app`, `env` (dev or prod), `tier` (frontend or backend) and `version`.

**The left side: a selector.** The selector says "find every object with the label `app = web`". The arrows go to Pods 1, 2 and 3 because they match. Pod 4 is skipped because its label is different.

## Why not use names?

Remember Figure 2: Pods die and are replaced with new names. If Kubernetes tracked Pods by name, it would lose them every time. But the new Pod gets the same label, so the selector finds it automatically. Labels are stable, names are not.

## Where selectors are used

- A **Deployment or ReplicaSet** uses a selector to know "these Pods are mine, and I must keep 3 alive".
- A **Service** (Chapter 3) uses a selector to know "send user traffic to these Pods".
- You can use them yourself to filter, for example "show me all Pods in `env = prod`".

### Common beginner mistake

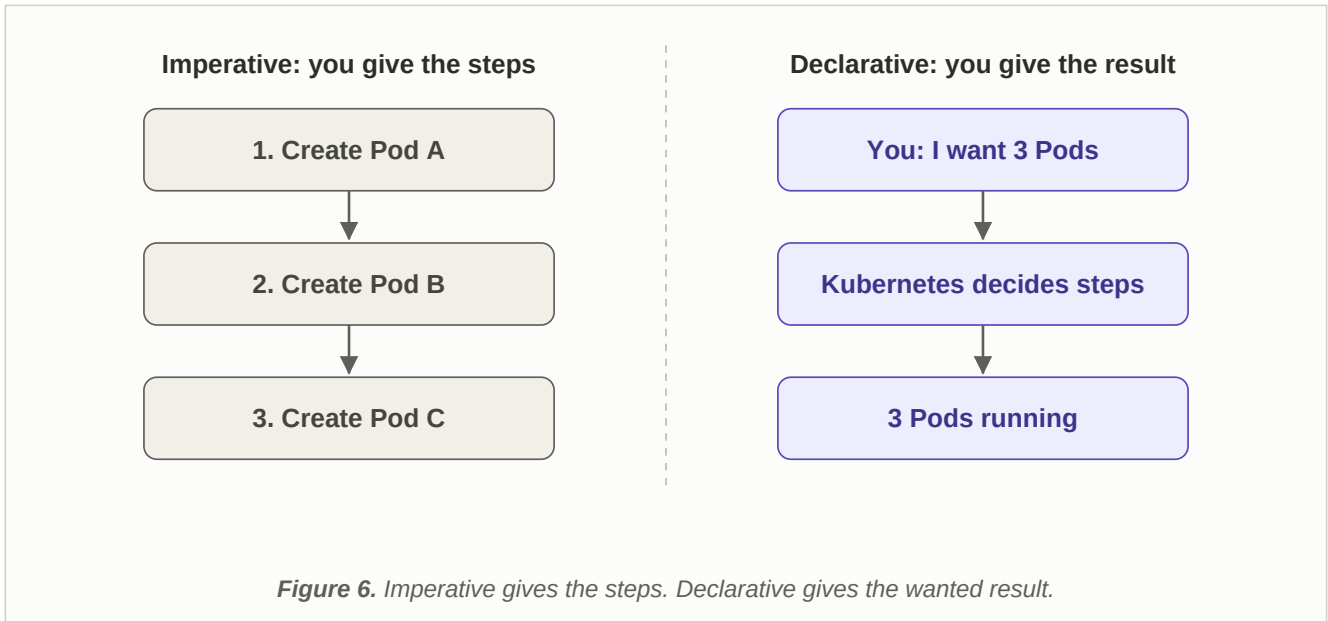
The label in the selector must exactly match the label on the Pod. If one says `app = web` and the other says `app = website`, nothing connects, and the Deployment or Service will not work.

### Real-life example

In a school, every student wears a badge with their class, such as `class = 5A`. The teacher says "all students with badge 5A, come to the front." The teacher does not need to know names, and it still works when a new student joins class 5A.

## 6. YAML manifests and declarative configuration

There are two ways to tell Kubernetes what to do. In the **imperative** way you give commands step by step. In the **declarative** way you describe the result you want, and Kubernetes makes it true.



### Explaining Figure 6

**Left: Imperative (you give the steps).** You act like a boss who controls every step: "Create Pod A. Now create Pod B. Now create Pod C." You must remember what you already did. If Pod B dies later, nobody replaces it unless you notice and do it again. This is fine for quick tests, but hard to manage in real projects.

**Right: Declarative (you give the result).** You only say what you want at the end: "I want 3 Pods." Kubernetes compares your wish with what exists and decides the steps by itself: create, delete or update whatever is needed. If a Pod dies, Kubernetes sees a difference and fixes it without your help (Figure 2). You can apply the same file 100 times and get the same result, because Kubernetes only changes what is different.

### Where do you write your wish? In a YAML manifest.

A **manifest** is a text file written in YAML. Every manifest answers four questions:

| Part       | Question it answers                                           |
|------------|---------------------------------------------------------------|
| apiVersion | Which version of the Kubernetes API do I use?                 |
| kind       | What type of object is this (Pod, Deployment, Namespace)?     |
| metadata   | What is its name, and which labels or Namespace does it have? |
| spec       | What exactly do I want (how many copies, which image)?        |

### Why teams prefer declarative

- 1 The file can be saved in **Git**, so everyone can see who changed what, and when.
- 2 You can rebuild an entire cluster from the files.

- 3 It is safer, because the file is the single source of truth.

#### Real-life example

Imperative is telling the kitchen "Take flour, add water, mix, bake 20 minutes at 250 degrees." Declarative is telling the waiter "One margherita pizza, please." The kitchen knows the steps, and you only care about the result.

## Chapter 2 summary

| Figure | Main idea                                                                      |
|--------|--------------------------------------------------------------------------------|
| 1      | A Deployment creates a ReplicaSet, which creates Pods, which run containers    |
| 2      | Pods are temporary, and the ReplicaSet replaces dead ones with new ones        |
| 3      | A Deployment updates Pods slowly for zero downtime, and can roll back          |
| 4      | Namespaces are virtual folders that separate teams and environments            |
| 5      | Labels and selectors connect objects, because names keep changing              |
| 6      | Declarative means you describe the wanted result, and Kubernetes makes it true |

### Check yourself

Try to answer these in your own words, without looking at the notes:

- 1 Which object creates the ReplicaSet, and which object creates the Pods?
- 2 In Figure 2, why is the new Pod called D and not B?
- 3 In Figure 3, why do users not see downtime? How does a rollback work?
- 4 Give two reasons to use Namespaces.
- 5 In Figure 5, why does Kubernetes use labels and not Pod names?
- 6 What is the difference between imperative and declarative? Which is better for teams, and why?

#### Next: Chapter 3, Networking

You will learn how a **Service** uses the selectors from Figure 5 to send traffic to Pods, and how Pods talk to each other across different worker nodes.